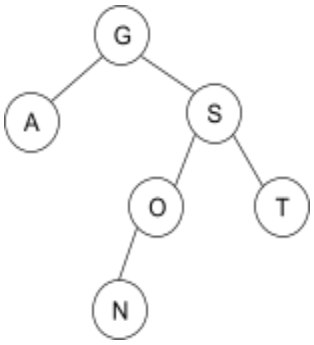
American Computer Science League

2024-2025 ● Contest 4: Binary Tree Arrays ● Senior Division

PROBLEM: The *Binary Search Tree* built from the letters **GASTON** is shown on the right. One way to represent this data structure is to use 3 arrays:

- an array of the *keys* in the order that they appear in the string
- an array of the position of each key's *left* child
- an array of the position of each key's *right* child



For each key, find where it must go in the tree and update the value of the parent's left or right array with the key's location. A value of -1 in the left or right array indicates that the node does not have that child. Here are the 3 arrays that are created from the letters **GASTON**:

	0	1	2	3	4	5	…
keys	**G**	**A**	**S**	**T**	**O**	**N**	
left	1	-1	4	-1	5	-1	
right	2	-1	3	-1	-1	-1	

In this program, you will be given a string to create a binary search tree from its letters. Once the tree is built using these arrays, find the following pieces of information for each tree: the number of nodes with only a left child, the number of nodes with only a right child, the number of leaf nodes, the internal path length, and the external path length. Output these 5 values as a string with each number separated by a single space.

EXAMPLE:

Input	`GASTON`
Output	`1 0 3 9 21`
Explanation	Initialize all elements in the left and right arrays to -1. The letter **G**: set key[0]='G'. The letter **A**: set key[1]='A'. A is the left child of G, so left[0]=1. The letter **S**: set key[2]='S'. S is the right child of G, so right[0]=2. The letter **T**: set key[3]='T'. T is the right child of S, so right[2]=3. The letter **O**: set key[4]='O'. O is the left child of S, so left[2]=4. The letter **N**: set key[5]='N'. N is the left child of O, so left[4]=5. The number of nodes with only a left child is 1 (letter **O**); the number of nodes with only a right child is 0; and the number of nodes that are leaves is 3 (**A**, **T**, **N**). The internal path length is the sum of the depths of each node, so the sum is 0 + 1 + 1 + 2 + 2 + 3 = 9. The external path length is the sum of the depths of

	the nodes that don't yet exist in the tree which is 2 + 2 (from node **A**) + 3 + 3 + 3 (from nodes **O** and **T**) + 4 + 4 (from node N) = 21. The output is `1 0 3 9 21`.

INPUT: Input one string that are all uppercase alphabetic characters and no more than 20 characters long.

OUTPUT: Output a string of numeric values, each separated by a single space, for each of the above 5 characteristics of the binary search tree.

SAMPLE INPUT:

```
1. GASTON
2. JASMINE
3. PETERPAN
4. RAPUNZEL
5. PINOCCHIO
6. MICKEYMOUSE
```

SAMPLE OUTPUT:

```
1. 1 0 3 9 21
2. 2 2 2 12 26
3. 3 0 3 14 30
4. 2 3 2 18 34
5. 2 2 3 22 40
6. 3 3 3 25 47
```

American Computer Science League

2024-2025 ● Contest 4: Binary Tree Arrays ● Senior Division

TEST DATA

TEST INPUT:

```
1. CINDERELLA
2. MALEFICENT
3. DONALDDUCK
4. BUZZLIGHTYEAR
5. SLEEPINGBEAUTY
6. A
```

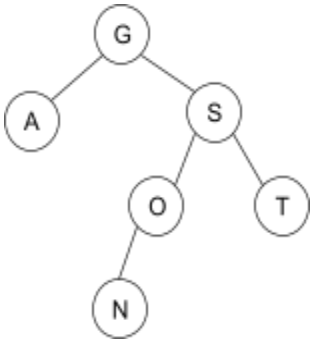
TEST OUTPUT:

```
1. 2 1 4 23 43
2. 1 4 3 27 47
3. 4 1 3 22 42
4. 4 0 5 37 63
5. 3 0 6 37 65
6. 0 0 1 0 2
```

American Computer Science League

2024-2025 ● Contest 4: Binary Tree Arrays ● Senior Division

PROBLEM STATEMENT:



The *Binary Search Tree* built from the letters **GASTON** is shown on the right. One way to represent this data structure is to use 3 arrays:

- an array of the *keys* in the order that they appear in the string
- an array of the position of each key's *left* child
- an array of the position of each key's *right* child

For each key, find where it must go in the tree and update the value of the parent's left or right array with the key's location. A value of -1 in the left or right array indicates that the node does not have that child. Here are the 3 arrays that are created from the letters **GASTON**:

	0	1	2	3	4	5	…
keys	**G**	**A**	**S**	**T**	**O**	**N**	
left	1	-1	4	-1	5	-1	
right	2	-1	3	-1	-1	-1	

In this program, you will be given a string to create a binary search tree from its letters. Once the tree is built using these arrays, find the following pieces of information for each tree: the number of nodes with only a left child, the number of nodes with only a right child, the number of leaf nodes, the internal path length, and the external path length. Output these 5 values as a string with each number separated by a single space.

EXAMPLE:

Input:

```
GASTON
```

Output:

```
1 0 3 9 21
```

Explanation:

Initialize all elements in the left and right arrays to -1.

The letter **G**: set key[0]='G'.
The letter **A**: set key[1]='A'. A is the left child of G, so left[0]=1.
The letter **S**: set key[2]='S'. S is the right child of G, so right[0]=2.
The letter **T**: set key[3]='T'. T is the right child of S, so right[2]=3.
The letter **O**: set key[4]='O'. O is the left child of S, so left[2]=4.
The letter **N**: set key[5]='N'. N is the left child of O, so left[4]=5.

The number of nodes with only a left child is 1 (letter **O**); the number of nodes with only a right child is 0; and the number of nodes that are leaves is 3 (**A**, **T**, **N**). The internal path length is the sum of the depths of each node, so the sum is 0 + 1 + 1 + 2 + 2 + 3 = 9. The external path length is the sum of the depths of the nodes that don't yet exist in the tree which is 2 + 2 (from node **A**) + 3 + 3 + 3 (from nodes **O** and **T**) + 4 + 4 (from node **N**) = 21. The output is `1 0 3 9 21`.

TASK:

Complete the function **analyzeTree**

- The function has 1 parameter: a string, *word*, that is used to build the binary search tree
- The function returns a string of 5 numeric values which are identified above, each separated by a single space

You may create additional functions that are called from **analyzeTree** if needed in solving the problem.

CONSTRAINTS:

The word will be all uppercase letters and no more than 20 characters in length.

DATA PROVIDED:

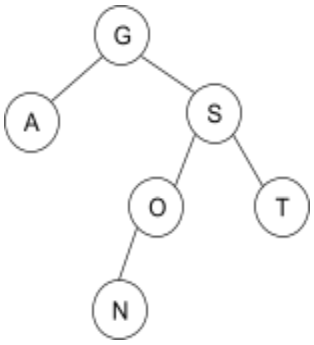
There are 6 sets of Sample Data for debugging and 6 sets of Test Data for scoring. The test cases will vary in difficulty. You should create sample data of your own to fully test your program.

American Computer Science League

2024-2025 ● Contest 4: Binary Tree Arrays ● Senior Division

问题：由字符串 GASTON 构建的*二叉搜索树*如右图所示。该数据结构可通过以下三个数组表示：

- 一个按字符串中出现顺序存储*键*的数组
- 一个数组记录每个键*左*子节点的位置
- 一个数组记录每个键*右*子节点的位置



插入新键时，需确定其在树中的位置，并将其父节点左子数组或右子数组的值更新为新键的位置。若左子数组或右子数组中的值为 -1，则表示该节点对应的子节点不存在。

以下是由字符串 GASTON 生成的三个数组：

	0	1	2	3	4	5	…
键	**G**	**A**	**S**	**T**	**O**	**N**	
左子节点	1	-1	4	-1	5	-1	
右子节点	2	-1	3	-1	-1	-1	

本程序将提供一个字符串，你需要根据其字母来构建二叉搜索树。树构建完成后，请为每棵树找出以下信息：仅含左子节点的节点数、仅含右子节点的节点数、叶节点数、内部路径长度和外部路径长度。将这 5 个数值以字符串形式输出，各数值之间用单个空格分隔。

示例：

输入	GASTON
输出	1 0 3 9 21
解析	将左子节点数组和右子节点数组的所有元素初始化为 -1。 字母 **G**：设置 key[0]='G'。 字母 **A**：设置 key[1]='A'。A 是 G 的左子节点，故 left[0]=1。 字母 **S**：设置 key[2]='S'。S 是 G 的右子节点，故 right[0]=2。 字母 **T**：设置 key[3]='T'。T 是 S 的右子节点，故 right[2]=3。 字母 **O**：设置 key[4]='O'。O 是 S 的左子节点，故 left[2]=4。 字母 **N**：设置 key[5]='N'。N 是 O 的左子节点，故 left[4]=5。 仅含左子节点的节点数为 1（字母 **O**）；仅含右子节点的节点数为 0；叶节点数为 3 (**A**, **T**, **N**）。内部路径长度为各节点深度之和，计算结果为 0 + 1 + 1 + 2 + 2 + 3 = 9。

	外部路径长度为树中不存在节点的深度之和，计算过程为：2 + 2（来自节点 **A**） + 3 + 3 + 3（来自节点 **O** 和节点 **T**） + 4 + 4（来自节点 **N**）= 21。 输出结果为 `1 0 3 9 21`。

输入：输入一个全由大写字母组成的字符串，长度不超过 20 个字符。

输出：输出一串数值，每个数值之间用单个空格分隔，分别表示上述二叉搜索树的 5 个特征。

样例输入：

```
1. GASTON
2. JASMINE
3. PETERPAN
4. RAPUNZEL
5. PINOCCHIO
6. MICKEYMOUSE
```

样例输出：

```
1. 1 0 3 9 21
2. 2 2 2 12 26
3. 3 0 3 14 30
4. 2 3 2 18 34
5. 2 2 3 22 40
6. 3 3 3 25 47
```

测试数据

测试输入：

```
1. CINDERELLA
2. MALEFICENT
3. DONALDDUCK
4. BUZZLIGHTYEAR
5. SLEEPINGBEAUTY
6. A
```

测试输出：

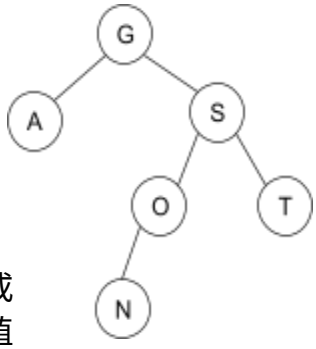
```
1. 2 1 4 23 43
2. 1 4 3 27 47
3. 4 1 3 22 42
4. 4 0 5 37 63
5. 3 0 6 37 65
6. 0 0 1 0 2
```

American Computer Science League

2024-2025 ● Contest 4: Binary Tree Arrays ● Senior Division

问题陈述:

由字符串 GASTON 构建的*二叉搜索树*如右图所示。该数据结构可通过以下三个数组表示：



- 一个按字符串中出现顺序存储*键*的数组
- 一个数组记录每个键*左*子节点的位置
- 一个数组记录每个键*右*子节点的位置

插入新键时，需确定其在树中的位置，并将其父节点左子数组或右子数组的值更新为新键的位置。若左子数组或右子数组中的值为 -1，则表示该节点对应子节点不存在。

以下是由字符串 GASTON 生成的三个数组：

	0	1	2	3	4	5	…
键	**G**	**A**	**S**	**T**	**O**	**N**	
左子节点	1	-1	4	-1	5	-1	
右子节点	2	-1	3	-1	-1	-1	

本程序将提供一个字符串，你需要根据其字母来构建二叉搜索树。树构建完成后，请为每棵树找出以下信息：仅含左子节点的节点数、仅含右子节点的节点数、叶节点数、内部路径长度和外部路径长度。将这 5 个数值以字符串形式输出，各数之间用单个空格分隔。

示例:

输入：

```
GASTON
```

输出：

```
1 0 3 9 21
```

解析：

将左子节点数组和右子节点数组的所有元素初始化为 -1。

字母 **G**：设置 key[0]=’G’。
字母 **A**：设置 key[1]=’A’。A 是 G 的左子节点，故 left[0]=1。
字母 **S**： 设置 key[2]=’S’。S 是 G 的右子节点，故 right[0]=2。
字母 **T**：设置 key[3]=’T’。T 是 S 的右子节点，故 right[2]=3。
字母 **O**：设置 key[4]=’O’。O 是 S 的左子节点，故 left[2]=4。
字母 **N**：设置 key[5]=’N’。N 是 O 的左子节点，故 left[4]=5。

仅含左子节点的节点数为 1（字母 **O**）；仅含右子节点的节点数为 0 ；叶节点数为 3 (**A**, **T**, **N**）。内部路径长度为各节点深度之和，计算结果为 0 + 1 + 1 + 2 + 2 + 3 = 9。外部路径长度为树中不存在节点的深度之和，计算过程为：2 + 2（来自节点 **A**） + 3 + 3 + 3（来自节点 **O** 和节点 **T** ） + 4 + 4（来自节点 **N** ） = 21。
输出结果为 `1 0 3 9 21`。

任务:
完成函数 **analyzeTree**

- 该函数有 1 个参数：一个字符串，*word*，用于构建二叉搜索树。
- 该函数返回一个字符串，包含 5 个用单个空格分隔的数值，这些数值如上文所述。

如有需要，你可以构建额外的函数供 **analyzeTree** 调用来解答此问题。

约束条件：

该字符串将全部由大写字母组成，且长度不超过 20 个字符。

数据说明:
包含 6 组用于调试的样例数据和 6 组用于评分的测试数据。测试案例难度各不相同。您应该自行创建样例数据以充分验证程序。